

WEPAStacks Inbound Congestion Prediction and Single-Drone Routing

An MLP-Filtered Ant Colony Optimisation Framework

Undergraduate Machine Learning Project Report

September 2026

Abstract

This project studies whether recent inbound activity can support warehouse-buffer congestion prediction and proactive pickup decisions. The provided WEPAStacks delivery records are aggregated into 15-minute intervals for four inbound points. A multilayer perceptron (MLP) maps seven current-and-past features to a sigmoid risk score for each warehouse. The simulated target is positive when the current stock plus arrivals during the next 60 minutes reaches 80% of warehouse capacity under a no-drone counterfactual.

The updated routing design deliberately separates prediction from optimisation. Warehouses with goods are passed to ant colony optimisation (ACO) only when their MLP score is at least 0.70. ACO then uses pheromone and distance—not the MLP score—to construct one route for one simulated drone. Its objective first penalises unvisited active warehouses and then route distance. The repository contains held-out prediction, ablation, calibration, multi-seed, and full-period routing artifacts for this implementation. Their interpretation remains limited: all positive validation and test labels occur at Warehouse 1, and no evaluated interval contains more than one active warehouse. The routing evidence therefore evaluates MLP-triggered dispatch timing, not ACO route-order quality.

Keywords: warehouse congestion, multilayer perceptron, time-series classification, ant colony optimisation, drone routing, WEPAStacks

Contents

1	Introduction	3
2	Part A — Task Definition	3
2.1	Objective and Motivation	3
2.2	Project Scope and Data Assumptions	3
2.3	Training and Deployment I/O	4
2.4	Research Question and Success Criteria	5
3	Part B — Model, Algorithm, and System Structure	5
3.1	Dataset and Preprocessing	5
3.1.1	Dataset Structure and Quality	5
3.1.2	Aggregation and Zero Intervals	6
3.1.3	Rolling Features, Queue Simulation, and Leakage Control	6
3.2	End-to-End System Overview	7
3.3	MLP architecture	7

3.4	Loss, optimiser, and class imbalance	7
3.5	Framework and implementation choices	8
3.6	MLP filter and single-drone ACO	8
3.7	Theory-to-code mapping	9
3.8	Reproducibility and Hyperparameter Configuration	9
3.9	Implementation Details and Observed Challenges	10
4	Part C — Model Evaluation and Refinement	10
4.1	Expected Model Behaviour	10
4.2	Evaluation Protocol and Metrics	11
4.3	Prediction Performance	11
4.4	Operational Performance	12
4.5	Implementation Verification	13
4.6	Answer to the Research Question	13
4.7	Loss–Objective Mismatch	13
4.8	Interpretation and Trade-offs	14
4.9	Limitations	14
4.10	Recommended Refinements	14
5	Conclusion	15
6	Reproducibility Checklist	15
A	Configured Drone Experiment	16
B	Metric Definitions	16

1 Introduction

Inbound congestion occurs when arriving goods accumulate faster than a handling system can remove them. A useful decision-support system should distinguish current or near-term congestion from lower-risk states, and its pickup policy should respect distance, energy, and payload limits. This project investigates these linked but distinct tasks. The MLP estimates current-or-next-hour threshold risk. Its score is used once, as a filter. ACO receives only the warehouses that pass the filter and constructs a single-drone route from their current goods and distances.

The practical motivation is that a reactive fill rule uses only the present queue. It does not directly use recent arrival intensity, so it may activate later than a predictor that recognises a rising pattern. This motivates testing an MLP, but does not establish that machine learning is necessary. The prediction stage is therefore compared with both an always-negative rule and a current-fill rule. At the operational stage, MLP-ACO is compared with nearest-first, fullest-first, and ACO-current. The MLP is kept small because the input is tabular and its recent history is already summarised; its deployment suitability is not assumed.

The central research question is:

On the chronological held-out period, does filtering warehouses with an MLP risk threshold before single-drone ACO reduce overflow relative to current-fill routing, while respecting route-distance, energy, and payload constraints?

The prediction stage asks whether seven present-and-past features improve threshold classification over simple rules. The operational stage asks whether the resulting filter improves routing outcomes. A high classification score alone cannot answer the operational question.

2 Part A — Task Definition

2.1 Objective and Motivation

The proposed contribution is an MLP-filtered ACO pipeline for one drone. The MLP produces one score per warehouse. A hard threshold of 0.70 determines which non-empty warehouses are visible to ACO. The score is not reused as an urgency weight, projected-stock estimate, or ACO objective term. This separation makes the role of machine learning testable: it changes the candidate set, while ACO remains a distance-based route optimiser.

The report uses a drone-only queue definition. Goods leave a simulated warehouse only when the single drone collects them. Battery is restored at the beginning of every 15-minute interval, representing replacement or charging at the depot. The earlier service-rate and multi-drone designs are outside the scope of the current experiment. The training-state simulator and deployment simulator use the same warehouse capacities, single-drone configuration, per-interval battery policy, and queue-update equations.

2.2 Project Scope and Data Assumptions

The project distinguishes fields in the provided WEPASacks records from experimental assumptions introduced by the simulation. The supplied fields describe a temporal stream of inbound events; the current file does not include four geographically separated warehouses, drone specifications, a central depot, or measured congestion labels.

Table 1: Provided dataset information and simulated assumptions

Category		Used in the project	Interpretation
Provided	WEPAS-	Arrival time, inbound dock, SKU, batch, and week where available	The event records determine time-varying arrival demand in the experiment. The baseline model uses arrival time and dock; SKU and batch are excluded.
tacks fields			
Simulated	system	Queue length, historical drone pickups, warehouse capacities, and congestion at 80% capacity	These variables define a synthetic prediction target. They are not observed ground truth, and reproducibility must be confirmed from the final configuration and seed records.
state			
Simulated routing en-		Four satellite warehouses, planar coordinates, central depot, and one drone with payload, battery, energy, and route-distance limits	These values define the proposed routing experiment. One event is treated as one standardised cargo unit, not as a claim that a small drone carries a physical pallet.
vironment			

2.3 Training and Deployment I/O

Each training observation represents one dock at the end of a 15-minute interval. The input vector is

$$\mathbf{x}_{i,t} = [r_{i,t}, a_{i,t}, \bar{a}_{i,t}^{(60)}, d_{i,1}, d_{i,2}, d_{i,3}, d_{i,4}] \in \mathbb{R}^7, \quad (1)$$

where $r_{i,t}$ is the buffer fill ratio, $a_{i,t}$ is the current number of arrivals, $\bar{a}_{i,t}^{(60)}$ is the mean arrival count over the current and previous three intervals, and $d_{i,j}$ is a one-hot dock indicator. The feature definition is intended to restrict input values to information available at or before time t ; this property must be confirmed in the final leakage audit.

The binary training target assumes that no drone will collect goods during the next four intervals. It is

$$y_{i,t} = \mathbb{1} \left[\max_{h \in \{1,2,3,4\}} \left(A_{i,t} + \sum_{s=1}^h a_{i,t+s} \right) \geq 0.8K_i \right], \quad (2)$$

where $A_{i,t}$ is the available stock before the current pickup decision and K_i is warehouse capacity. The configured capacities are 200 units for Warehouse 1 and 80 units for Warehouses 2–4. Future arrivals are required for historical label construction. The feature-building procedure is intended to exclude them from the input vector, and that separation must be verified before final reporting.

Table 2: Technical input and output specification

Phase	Interface	Valid input	Output
Training-label construction	One warehouse–interval sample	A seven-value vector in the fixed order shown above; the three numeric features are scaled from training statistics and the four dock indicators remain binary	One binary ground-truth label indicating whether no-drone stock reaches 80% capacity during the next four intervals; 0.80 is not an MLP output threshold
MLP inference	One simultaneous four-warehouse state	A 4×7 float32 feature matrix containing current fill ratio, current arrivals, trailing four-interval mean, and dock identity	A vector $\hat{\mathbf{p}}_t \in [0, 1]^4$ containing one continuous sigmoid score per warehouse; the MLP does not output a fixed value of 0.80
Pre-ACO filter	Current goods and four MLP scores	Warehouse i is retained only if $A_{i,t} > 0$ and $\hat{p}_{i,t} \geq 0.70$	A sorted mapping from active warehouse identifier to currently available goods
ACO deployment	Filtered goods, a 5×5 distance matrix, one drone state, ACO parameters, and seed	Non-negative goods; valid depot/warehouse nodes; positive payload, energy, battery, and maximum-distance limits	One depot-to-depot route, proportional pickup amounts, route distance, energy used, remaining active goods, objective value, and validation status

2.4 Research Question and Success Criteria

Table 3: Questions and pre-declared evidence required to answer them

Question	Evidence	Success criterion
Prediction	PR–AUC, ROC–AUC, precision, recall, F1, confusion matrix, and per-warehouse support on chronological validation and test data	Compare with always-negative and current-fill baselines; do not generalise positive-class performance to a warehouse without positive support.
Primary routing	Overflow, collected units, energy, distance, sorties, active-warehouse visits, runtime, and independently recomputed constraint checks	Compare MLP–ACO with nearest-first, fullest-first, and ACO-current on identical held-out arrivals.
Stability	Repeated ACO runs on identical states and arrival streams	Use the ten configured seeds for stochastic policies and report mean and standard deviation; label any shorter run as diagnostic.

3 Part B — Model, Algorithm, and System Structure

3.1 Dataset and Preprocessing

3.1.1 Dataset Structure and Quality

The supplied WEPASStacks documentation describes 89 days of operations in a finished-goods warehouse and three source files: a layout file, an order-event file, and an initial-fill file. The current implementation does not consume all three files directly. It uses a cleaned CSV derived from delivery events at inbound points 1–4; retrieval events, outbound points, the original layout, and initial inventory are outside the present learning interface.

The current cleaned dataset contains 205,258 inbound events and nine fields: arrival time, relative day, hour, minute, 15-minute time bin, dock, SKU, batch, and week. It covers relative days 2–89. Preliminary checks recorded by the current preprocessing workflow did not identify missing values, invalid dock identifiers, negative arrival times, or fully duplicated rows. The workflow also reported agreement between the derived day, hour, minute, and time-bin fields and arrival time. These checks should be rerun and independently reviewed before submission. There are 4,763 repeated arrival-time and dock combinations; they are currently retained because multiple distinct cargo events may occur at the same dock in the same second.

An evident data-quality concern is representativeness. Dock 1 contains 93.26% of all events, while Docks 2–4 disappear early in the timeline. In the current cleaned file, events after day 39 are assigned only to Dock 1. The reason for this pattern is not established by the dataset and should not be inferred without additional metadata.

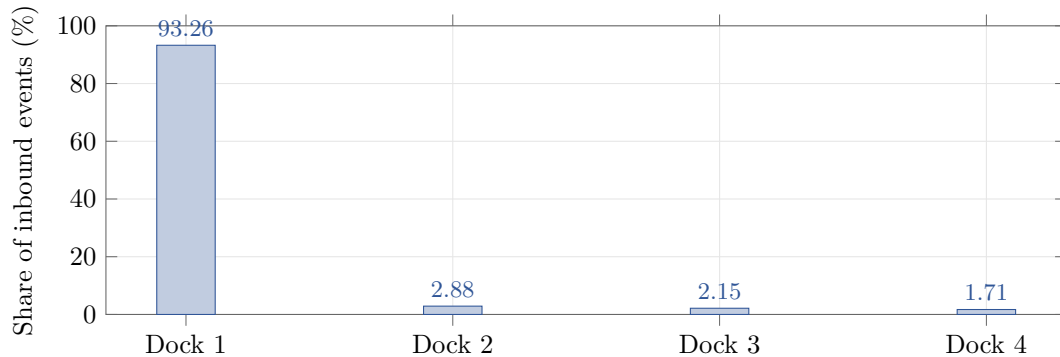


Figure 1: Inbound-event distribution by dock. Class weights may compensate for label imbalance, but they do not resolve unequal dock coverage.

Table 4: Temporal coverage and arrival distribution by dock

Dock	Events	Share	First day	Last day	Active days	Zero intervals
1	191,428	93.26%	2	89	88	20.25%
2	5,902	2.88%	2	39	12	92.65%
3	4,421	2.15%	2	18	8	94.78%
4	3,507	1.71%	2	11	5	95.83%

3.1.2 Aggregation and Zero Intervals

Arrival time is divided by 900 seconds and rounded down to obtain a 15-minute time bin. Events are grouped by time bin and dock. A complete Cartesian grid is then created from all 8,395 consecutive time bins and the four docks, producing 33,580 dock–interval rows before rolling and future-horizon exclusions. Missing counts are filled with zero. This step is necessary because a missing event row may mean that an active dock received no goods during the interval.

The same operation also creates a limitation. For Docks 2–4, a zero after the final recorded event may mean either zero demand or missing/inactive coverage. The dataset has no `dock_active` field, so the pipeline cannot distinguish these cases. The report therefore treats late zero rows for Docks 2–4 cautiously.

3.1.3 Rolling Features, Queue Simulation, and Leakage Control

The 60-minute rolling feature is the mean of arrivals in the current and previous three intervals. The first three rows for each dock are discarded because a complete window is unavailable. Historical training states are generated chronologically by a fixed fullest-first drone policy. The policy is designed to read current available goods, current battery, capacities, and distances without using future arrivals or MLP predictions. This restriction remains part of the final leakage audit. The state update is

$$A_{i,t} = Q_{i,t} + a_{i,t}, \quad (3)$$

$$R_{i,t} = \max(0, A_{i,t} - P_{i,t}), \quad (4)$$

$$O_{i,t} = \max(0, R_{i,t} - K_i), \quad (5)$$

$$Q_{i,t+1} = \min(K_i, R_{i,t}), \quad (6)$$

where $P_{i,t}$ is the quantity collected by the single drone. There is no separate service-rate term. Battery is reset before each simulated interval. The feature `buffer_fill_ratio` is calculated from $A_{i,t}$ before pickup, so it can exceed 1 when arrivals temporarily exceed capacity; its current modelling range is 0.20 to 1.55.

The final four rows for each dock are discarded because they do not have a complete future label horizon. The resulting modelling table contains 33,552 observations. The pipeline attempts to reduce leakage risk by using current and past arrivals as features, using a fixed non-ML historical policy, fitting the scaler and class weights on training data, inserting one-hour gaps at split boundaries, choosing the classification threshold on validation data, and reserving test data for final evaluation. These design choices reduce obvious leakage paths but do not replace an independent audit of feature timestamps, state updates, and split boundaries.

Table 5: Chronological data split

Split	Time range	Observations	Positive	Positive rate
Training	Days 2–60 with horizon gap	22,628	6,291	27.80%
Validation	Days 61–74 with horizon gap	5,360	965	18.00%
Test	Days 75–89	5,532	1,099	19.87%

3.2 End-to-End System Overview

Figure 2 shows the single-drone workflow. A fixed fullest-first policy produces historical queue states for supervised learning. During deployment, the MLP scores the four current states, the 0.70 filter removes low-score or empty warehouses, and ACO routes only over the retained set. Both stages use the same capacities, one-drone configuration, per-interval battery reset, and queue equations.

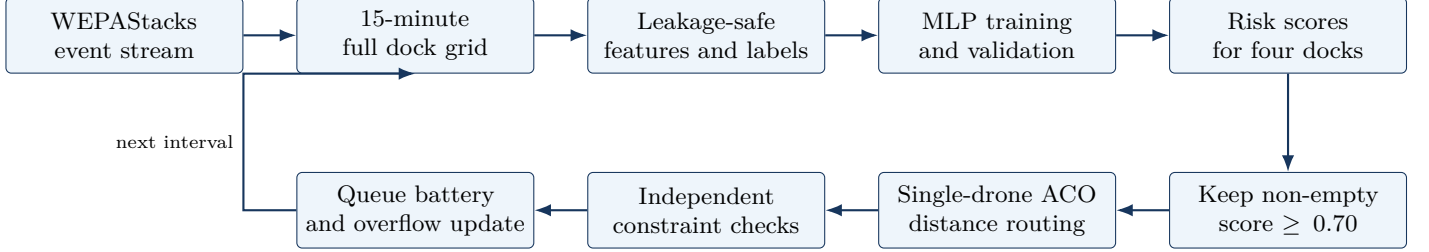


Figure 2: Updated data and decision flow. The MLP filters warehouses; its score is not used inside ACO.

3.3 MLP architecture

The network has two hidden layers. The first maps seven inputs to 32 ReLU units, and the second maps 32 values to 16 ReLU units. A single sigmoid output produces a risk score. Glorot uniform initialisation is used for the dense-layer weights, and all biases start at zero. No dropout layer is used in the drone-compatible implementation.

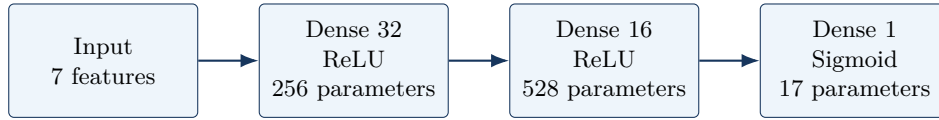


Figure 3: MLP architecture with 801 trainable parameters.

The forward pass can be written as

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \quad (7)$$

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2), \quad (8)$$

$$\hat{p} = \sigma(\mathbf{w}_3^\top \mathbf{h}_2 + b_3). \quad (9)$$

This compact structure can represent nonlinear interactions between current fill, current arrivals, and recent arrival history without introducing a large parameter count. The taper from 32 to 16 units compresses the learned representation before the binary output, and the complete network has only 801 trainable parameters. ReLU is selected to reduce the saturation concerns associated with sigmoid hidden units, while the output sigmoid maps the final logit to a score in $(0, 1)$. These choices follow standard feed-forward neural-network practice [1, 2]. The widths were configured as a practical small-network baseline rather than selected by an exhaustive architecture search; therefore, the report does not claim that 32 and 16 units are optimal.

3.4 Loss, optimiser, and class imbalance

The network minimises weighted binary cross-entropy:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{n=1}^N w_{y_n} [y_n \log \hat{p}_n + (1 - y_n) \log(1 - \hat{p}_n)]. \quad (10)$$

Balanced weights computed from the 22,628 training labels are $w_0 = 0.6925$ and $w_1 = 1.7984$. They make an error on the less frequent positive class more costly during training. This addresses label-frequency imbalance in the loss,

but it does not correct unequal temporal coverage across warehouses. Binary cross-entropy is used because the model has one binary target and a sigmoid output; it is a differentiable training surrogate, not the final operational objective.

For every mini-batch, Keras first evaluates the forward equations above and the weighted loss in Eq. (10). TensorFlow automatic differentiation then applies the chain rule through the sigmoid, dense layers, and ReLU activations to obtain $\nabla_{\theta} \mathcal{L}_{\text{BCE}}$. Adam maintains first- and second-moment estimates of these gradients and updates the weights using learning rate 10^{-3} , $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-7}$ [3]. Thus, backpropagation is performed by the framework rather than by a handwritten gradient routine, but it optimises the stated mathematical loss. Adam is used as a stable practical default for mini-batch training; the current evidence does not establish that it is better than every alternative optimiser for this dataset.

Training uses mini-batches of 256 and at most 100 epochs. A batch of this size reduces update noise and keeps the number of updates manageable for the 22,628 training samples. Early stopping monitors validation loss with patience eight and restores the best weights, limiting unnecessary epochs after validation performance stops improving. These settings were preconfigured engineering choices, not the result of a complete hyperparameter search.

3.5 Framework and implementation choices

pandas and NumPy implement event cleaning, 15-minute aggregation, temporal state updates, and array operations. scikit-learn supplies training-only feature scaling and evaluation metrics. TensorFlow/Keras is used for the MLP because it provides dense layers, automatic differentiation, class-weighted mini-batch training, early stopping, and model persistence in one reproducible interface. At deployment, the saved scaler and feature order are reloaded with the model so that inference uses the same representation as training.

ACO is implemented directly in Python rather than imported as a general routing solver. This makes the experiment-specific transition rule, return-to-depot feasibility check, unvisited-node penalty, and proportional pickup rule explicit. Route validation is kept separate from route construction so that payload, distance, energy, and node-eligibility constraints are recomputed independently. This separation reduces the chance that the optimiser can validate its own mistake. It does not make ACO necessary for a four-warehouse instance; exact enumeration remains a practical reference method.

3.6 MLP filter and single-drone ACO

At interval t , available goods are $A_{i,t} = Q_{i,t} + a_{i,t}$. For MLP-ACO, the active set is

$$\mathcal{S}_t = \{i \mid A_{i,t} > 0 \wedge \hat{p}_{i,t} \geq 0.70\}. \quad (11)$$

Warehouses outside $\mathcal{S}_t$ are absent from the ACO input and cannot appear in its route. For nearest-first, fullest-first, and ACO-current, the active set instead uses the current-fill rule $A_{i,t}/K_i \geq 0.80$. This is the only difference between ACO-current and MLP-ACO; both use the same route optimiser.

Each ant begins at depot 0 and maintains the travelled distance. A candidate warehouse j is feasible only if the route can visit it and still return to the depot:

$$D_{\text{proj}} = D_{\text{so far}} + d_{ij} + d_{j0}, \quad (12)$$

$$D_{\text{proj}} \leq D_{\text{max}}, \quad (13)$$

$$D_{\text{proj}}e \leq B_{\text{available}}, \quad (14)$$

where e is energy per kilometre. Among feasible unvisited warehouses, the transition probability is

$$P(i \rightarrow j) = \frac{\tau_{ij}^{\alpha} (1/(\varepsilon + d_{ij}))^{\beta}}{\sum_{k \in \mathcal{F}} \tau_{ik}^{\alpha} (1/(\varepsilon + d_{ik}))^{\beta}}. \quad (15)$$

The MLP score and amount of goods do not appear in this transition rule. The implementation uses $\alpha = 1$, $\beta = 2$, 30 ants, at most 100 iterations, evaporation 0.10, and a 20-iteration stall limit.

For route R , the objective is

$$J(R) = 1000 |S_t \setminus V(R)| + D(R), \quad (16)$$

where $V(R)$ is the set of visited active warehouses. Since every feasible route is limited to 30 km, the penalty makes an additional active visit more important than any possible distance saving. Among routes visiting the same number of active warehouses, the shorter route wins. After each iteration, pheromone evaporates and the global-best route receives an increment proportional to $1/(1 + J)$.

After route construction, the 30-unit payload is divided in proportion to available goods at visited warehouses. If $C = \min(30, \sum_{j \in V(R)} A_{j,t})$, then

$$x_{i,t} = C \frac{A_{i,t}}{\sum_{j \in V(R)} A_{j,t}}, \quad i \in V(R). \quad (17)$$

The validator recomputes depot endpoints, repeated and filtered nodes, total distance, route energy, payload, and per-warehouse pickup. Goods not collected remain in the queue. Battery is logged after the route and restored before the next interval.

3.7 Theory-to-code mapping

Table 6: Mapping from mathematical concepts to concrete implementation entry points

Component	Mathematical or system role	Function or class
Interval grid	Aggregates events and inserts explicit zero-arrival rows for every dock and interval	<code>aggregate_intervals()</code> in <code>src/data_pipeline.py</code>
Historical state	Generates queue states with the fixed fullest-first historical policy	<code>simulate_historical_queue()</code> in <code>src/queue_simulator.py</code>
Target and split	Implements Eq. (2) and chronological split boundaries	<code>add_no_drone_labels()</code> and <code>chronological_masks()</code> in <code>src/feature_builder.py</code>
Forward and backward pass	Builds the $7 \rightarrow 32 \rightarrow 16 \rightarrow 1$ network; <code>model.fit()</code> invokes automatic differentiation and Adam on Eq. (10)	<code>_fit_mlp()</code> in <code>src/train_mlp_aco.py</code>
Risk inference	Restores the scaler and feature order, then evaluates the saved MLP with training disabled	<code>RiskPredictor.predict_frame()</code> and <code>predict_by_warehouse()</code> in <code>src/predict_risk.py</code>
Hard filter	Implements Eq. (11); excluded warehouses never enter the optimiser	<code>select_active_warehouses()</code> in <code>src/routing_utils.py</code>
ACO transition	Builds feasible routes using Eq. (15); the outer routine updates pheromone and tracks the best solution	<code>_construct_ant_solution()</code> and <code>optimize_aco()</code> in <code>src/aco_optimizer.py</code>
Objective and pickup	Implements Eq. (16) and proportional payload allocation	<code>calculate_route_objective()</code> and <code>allocate_pickups()</code> in <code>src/routing_utils.py</code>
Greedy baselines	Constructs comparable nearest-first and fullest-first routes	<code>nearest_first()</code> and <code>fullest_first()</code> in <code>src/baselines.py</code>
Independent validation	Recomputes payload, battery, route distance, visited-node, and demand checks	<code>validate_solution()</code> in <code>src/solution_validator.py</code>
Rolling deployment	Calls prediction, filtering, routing, queue updates, overflow accounting, and per-interval battery reset	<code>run_simulation()</code> in <code>src/rolling_simulation.py</code>

3.8 Reproducibility and Hyperparameter Configuration

The current experiment configuration specifies seed 42 for Python, NumPy, and TensorFlow. Deterministic TensorFlow operations are requested where supported. The recorded development environment lists Python 3.9.6,

TensorFlow 2.20.0, Keras 3.10.0, NumPy 2.0.2, pandas 2.3.3, and scikit-learn 1.6.1. These details should be captured again for the final run, and exact bitwise equality across hardware and library versions should not be assumed.

Table 7: Main MLP hyperparameters

Parameter	Value	Parameter	Value
Interval	15 minutes	Horizon	4 intervals
Capacities	200, 80, 80, 80	Congestion level	80% of capacity
Hidden units	32, 16	Trainable parameters	801
Learning rate	0.001	Batch size	256
Maximum epochs	100	Early-stopping patience	8
Training boundary	Day 60	Validation boundary	Day 74
Operational threshold	Fixed at 0.70	Validation F1 diagnostic	0.9313

The table records the implemented configuration rather than the result of a broad tuning study. The 60-minute horizon matches four 15-minute intervals, and the chronological boundaries prevent later observations from entering model fitting. The compact hidden layers, learning rate, batch size, and patience were selected as reasonable baseline values. Their suitability is assessed through held-out performance, a linear-model comparison, a warehouse-identity ablation, and repeated training seeds, but these checks do not prove hyperparameter optimality. A future architecture study would be required before describing this configuration as tuned.

3.9 Implementation Details and Observed Challenges

The main data challenge is temporal alignment. Future arrivals are required to construct the label but must not enter the seven input features. The implementation therefore builds labels after sorting each warehouse by time, removes rows without a complete future horizon, inserts gaps at split boundaries, and fits the scaler only on training rows. Chronological splitting is used instead of random splitting because neighbouring intervals are dependent and a random split would mix later operating states into model development.

Class weighting addresses the overall positive-label frequency, but it cannot repair the warehouse coverage shift. Warehouse 1 dominates inbound activity, while Warehouses 2–4 contain no positive validation or test cases. Per-warehouse support, the identity-feature ablation, and the linear baseline are therefore needed to prevent a high global score from being interpreted as evidence of equal performance at every warehouse.

The hard 0.70 filter creates a discontinuity between prediction and routing: scores of 0.699 and 0.700 can produce different candidate sets even though they are almost equal. In addition, weighted BCE optimises label prediction rather than overflow, distance, or energy. The downstream threshold sweep is consequently interpreted as a trade-off analysis, not proof that 0.70 is optimal. The score is also called a risk score rather than a calibrated probability because reliability varies across sparsely supported score ranges.

The routing implementation has a different limitation. Although ACO can construct a multi-stop route, the observed evaluation stream contains no interval with more than one active warehouse. Repeating ACO seeds on this stream tests reproducibility and constraints but not stochastic route ordering. With only four warehouses, exact enumeration is also cheap. A separate multi-active stress test and comparison with the exact solver are required before making a route-quality claim about ACO.

4 Part C — Model Evaluation and Refinement

4.1 Expected Model Behaviour

Given low current fill and low recent arrivals, the MLP should produce a lower score than it produces for a state near the congestion boundary with sustained arrivals. For a positive ground-truth label, the ideal score is close to one; for a negative label, it is close to zero. The implemented comparison uses weighted binary cross-entropy

during training and held-out ranking and threshold metrics during evaluation. Because the target includes stock that is already at the 80% capacity boundary, the reported metrics combine current-state detection with genuine below-threshold forecasting; this distinction is treated as a limitation rather than hidden by the aggregate score.

The congestion threshold of 0.80 is used only to construct the binary ground-truth label from warehouse capacity. The MLP itself outputs a continuous score between zero and one. At deployment, the fixed operational threshold converts this score into a routing decision: a score of 0.70 is included and 0.699 is excluded. A warehouse with no goods is excluded even when its score is high. The validation-optimal classification threshold is retained as a diagnostic and does not silently replace the configured routing threshold.

The desired downstream behaviour is a valid depot-to-depot route over only the retained warehouses. ACO may visit each warehouse at most once, the route must remain within 30 km and 30 energy units, and total pickup cannot exceed 30 cargo units. Actual output is therefore compared with prediction labels and baselines at the MLP stage, and with overflow, collection, distance, energy, active visits, and independent constraint checks at the routing stage.

4.2 Evaluation Protocol and Metrics

Accuracy alone is insufficient when class balance and temporal coverage differ across docks. Precision measures how many warnings are correct. Recall measures how many threshold cases are detected. F1 balances precision and recall. Balanced accuracy averages sensitivity and specificity. Average precision summarises the precision–recall curve [5]. The confusion matrix and per-dock support are essential because strong global performance can still reflect a dock-identity shortcut rather than general temporal prediction.

For routing, the metrics are overflow, collected goods, energy, distance, number of sorties, active-warehouse visit rate, runtime, and independently recomputed constraint violations. The primary comparison is MLP–ACO against nearest-first, fullest-first, and ACO-current. The final run covers the entire held-out period and all ten configured seeds for each stochastic policy.

4.3 Prediction Performance

The chronological split produced 22,628 training samples, 5,360 validation samples, and 5,532 test samples. Their positive rates were 27.80%, 18.00%, and 19.87%, respectively. Training stopped after 75 epochs. At the fixed operational threshold of 0.70, the held-out test confusion counts were $TN = 4418$, $FP = 15$, $FN = 50$, and $TP = 1049$. Table 8 reports the principal metrics.

Table 8: Held-out prediction results at threshold 0.70

Method	Bal. accuracy	Precision	Recall	F1	PR–AUC	ROC–AUC
Always negative	0.5000	0.0000	0.0000	0.0000	–	–
Current fill ≥ 0.80	0.8904	1.0000	0.7807	0.8769	–	–
MLP score ≥ 0.70	0.9756	0.9859	0.9545	0.9699	0.9938	0.9984

Across five training seeds, mean test PR–AUC was 0.99381 ± 0.00023 and mean F1 was 0.96976 ± 0.00068 , indicating low optimisation variance for the fixed split. Removing warehouse one-hot features slightly increased thresholded F1 from 0.9699 to 0.9704 but reduced PR–AUC from 0.9938 to 0.9826. Logistic regression achieved F1 0.9668 and PR–AUC 0.9938. Thus, the MLP gives a modest thresholded improvement over the linear model, while the warehouse identity input does not explain its performance gain.

The result is not equally supported across warehouses. All 1,099 positive test labels occur at Warehouse 1. Warehouses 2–4 each contribute 1,383 negative samples and no positive samples, so their recall, F1, and positive-class ranking performance cannot be estimated. Figure 4 makes this limitation explicit.

The Brier score was 0.0122, but the reliability diagram shows that several sparsely supported middle-score bins lie below the ideal diagonal. The sigmoid output is therefore retained as a risk score rather than asserted to be a

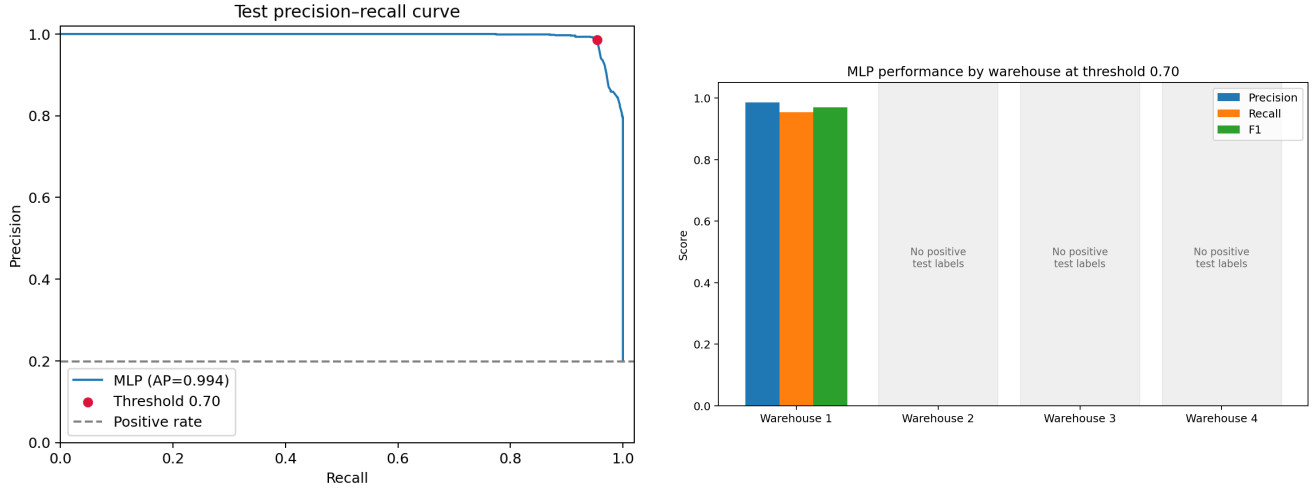


Figure 4: Held-out precision-recall evidence (left) and warehouse-specific support limitation (right).

fully calibrated probability. The validation-optimal F1 threshold was 0.9313, compared with the fixed operational threshold of 0.70. The fixed threshold has a similar validation F1 but makes more proactive routing decisions.

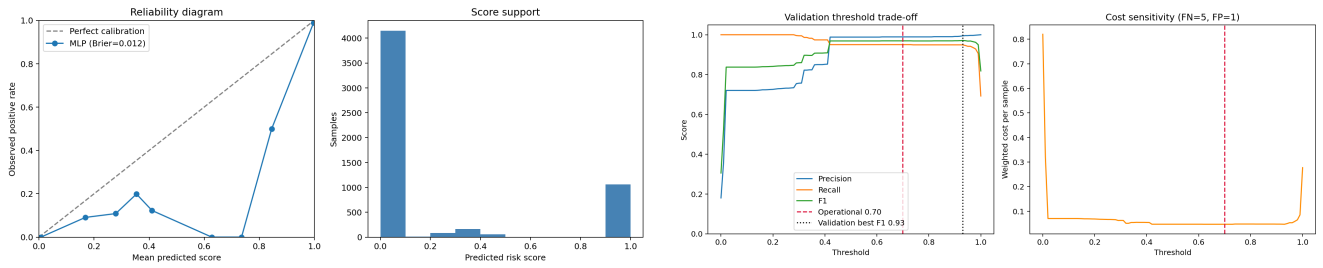


Figure 5: Reliability evidence (left) and validation threshold trade-off under an illustrative 5:1 false-negative cost (right).

4.4 Operational Performance

The complete held-out stream contains 1,387 time bins and 5,548 warehouse-interval rows. The stochastic policies were evaluated with all ten configured seeds. At threshold 0.70, MLP-ACO reduced mean overflow from 6,219 to 5,511 units relative to ACO-current, an 11.38% reduction, and collected 738 additional units. This required 270 additional distance and energy units, increasing both from 8,570 to 8,840. All independently checked payload, battery, route-distance, and route-validity constraints were satisfied.

Table 9: Full-period operational comparison

Policy	Overflow	Collected	Distance	Energy	Sorties
Nearest-first	6219	25710	8570	8570	857
Fullest-first	6219	25710	8570	8570	857
ACO-current	6219	25710	8570	8570	857
MLP-ACO	5511	26448	8840	8840	884

No policy encountered more than one active warehouse in any interval. Consequently, every ACO seed produced the same operational totals and the experiment evaluates MLP-triggered dispatch timing, not stochastic route ordering. This is conclusive negative evidence for the claimed multi-stop aspect under the current data and configuration.

The operational threshold sweep quantifies the loss-objective mismatch. Reducing the threshold from 0.70 to 0.30 reduced overflow from 5,511 to 4,445 units, but increased distance and energy from 8,840 to 10,620. Hence the threshold that best protects against overflow is not the threshold that minimises resource use; a business cost or Pareto criterion is required to select between them.

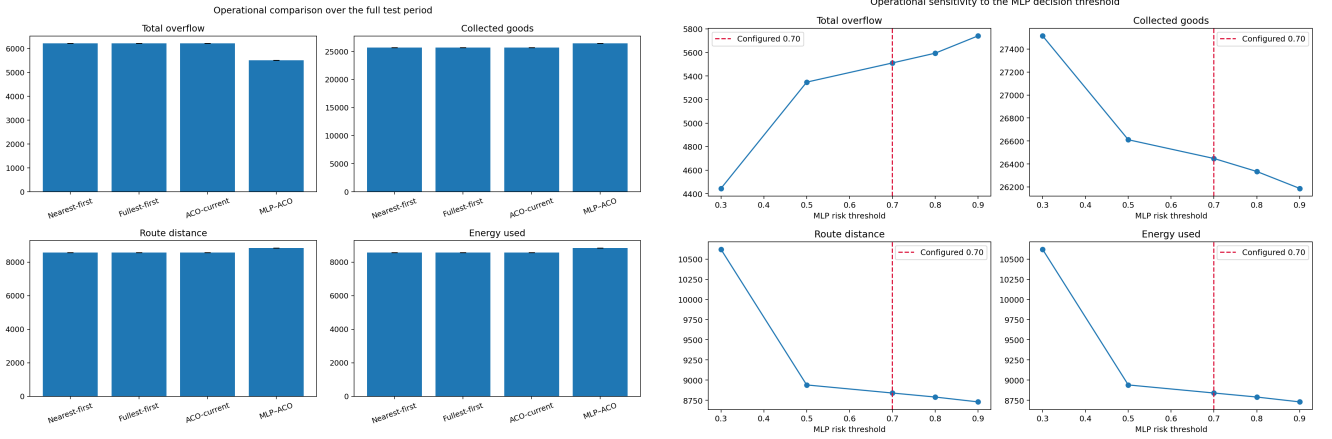


Figure 6: Full-period policy comparison (left) and downstream threshold sensitivity (right).

4.5 Implementation Verification

A fresh run of the current test suite completed 21 tests with no failures. The tests exercise the 0.70 inclusion boundary, exclusion of empty and filtered warehouses, depot endpoints, repeated-node rejection, payload and demand limits, energy and explicit distance limits, fixed-seed behaviour, per-interval battery reset, queue updates, and a small exact-reference comparison. This is evidence for those cases only; it is not a proof that every state or route is correct.

4.6 Answer to the Research Question

For the supported Warehouse 1 portion of the held-out data, the MLP improves congestion-warning F1 from 0.8769 for the current-fill baseline to 0.9699 and MLP filtering reduces simulated overflow by 11.38% relative to ACO-current at the configured threshold. These findings support proactive dispatch filtering. They do not demonstrate generalisation to positive cases at Warehouses 2–4, and they do not demonstrate ACO route-order optimisation because no interval contains multiple active warehouses.

4.7 Loss-Objective Mismatch

Weighted binary cross-entropy rewards agreement between the sigmoid score and the simulated binary label. The practical objective concerns overflow, collected goods, travel, energy, and missed service. The two objectives are related but not identical. Reducing BCE may move a score without changing which side of 0.70 it occupies, producing no routing change. Conversely, a small score change across 0.70 changes the ACO candidate set discontinuously.

The ACO objective also differs from the operational objective. Equation (16) maximises the number of active warehouses visited before minimising distance. It does not directly reward collected quantity or avoided overflow. For example, it can prefer visiting two small active warehouses over one high-volume warehouse when distance prevents visiting all three. Proportional pickup allocation is applied only after the route is chosen. These mismatches are tolerated to keep the algorithm transparent, but they must be monitored through overflow, collection, distance, and energy rather than through BCE or ACO objective value alone.

4.8 Interpretation and Trade-offs

The fixed operational threshold is 0.70, whereas validation F1 selected 0.9313. The operational sweep shows that lower thresholds trade additional distance and energy for lower overflow. Because the choice depends on the relative cost of these outcomes, 0.70 should be described as a configured policy point rather than an empirically optimal threshold. The sigmoid value is called a risk score because the reliability evidence does not establish calibration across the full range.

The single-drone design makes the algorithm easier to explain and validate. It also reduces the combinatorial difficulty: with at most four active warehouses there are only 24 complete visit orders before considering subsets. ACO remains a valid stochastic routing study, but an exact enumerator is computationally practical for this experiment and should be used more extensively as a reference.

4.9 Limitations

1. **Simulated target.** Queue length and congestion are generated from assumed capacities, drone behaviour, and a no-drone future scenario. The model predicts this construction, not measured warehouse congestion.
2. **Current-state contribution to the label.** The target starts from current available goods, so some positive cases are already at the 80% boundary. The task combines current-state detection with future forecasting.
3. **Warehouse coverage shift.** Warehouses 2–4 end early in the supplied event stream and have no positive validation or test labels.
4. **Ambiguous zero intervals.** Missing or inactive coverage is encoded in the same way as an active dock with zero arrivals.
5. **Dominant Dock 1.** Dock 1 contains 93.26% of inbound events, so the fitted shared model may be strongly influenced by its operating pattern.
6. **Warehouse identity ablation.** Removing one-hot identity slightly improved F1 but reduced PR–AUC, so identity is not the sole explanation for performance; interpretation remains limited by missing positive support outside Warehouse 1.
7. **Imperfect calibration.** The Brier score is low because most scores are near well-separated extremes, while sparsely supported middle-score bins deviate from the ideal reliability line.
8. **Limited feature set.** The baseline excludes hour-of-day, shift, robot state, downtime, service completions, and SKU mixture.
9. **Simplified single-drone physics.** Battery is restored every 15 minutes and the model omits flight duration, charging time, weather, no-fly zones, and collision avoidance.
10. **Hard filtering.** A score change from 0.699 to 0.700 can change routing eligibility even though the underlying states are almost identical.
11. **Objective mismatch.** ACO minimises unvisited active warehouses and distance, not expected overflow or collected quantity.
12. **No multi-stop routing evidence.** The complete run contains no interval with more than one active warehouse, so equal ACO results across seeds cannot establish route-order quality.
13. **Small routing instance.** With four warehouses, exact enumeration may be more appropriate than ACO unless the study is explicitly framed as a prototype for larger instances.

4.10 Recommended Refinements

One important improvement is to obtain data that distinguish an active dock with no arrivals from an inactive or unobserved dock. A `dock_active` field, operating calendar, and documented reason for the early termination of Warehouses 2–4 would support a more defensible temporal grid. Real queue length, service completions, incident logs, and confirmed capacities could then be used to assess Eq. (2) against an observed target.

Model evaluation should continue to preserve chronological splits and report support for every metric. The completed identity ablation should be extended by evaluating only cases whose current fill is below 0.80, thereby isolating genuine congestion onset. Calibration methods should be fitted on validation data and assessed on a new untouched test period; the present test period must not be reused for further model selection.

The completed threshold sweep should next be repeated on a validation simulation before selecting a deployment threshold. Candidate policies should also test a minimum pickup load. A decision-aware extension could replace the count-based penalty with estimated overflow cost, although this would make the algorithm more complex.

Future routing evaluation requires data or a scenario with simultaneous demand at multiple warehouses. Because there are at most four warehouses, every nontrivial active set can also be compared with the exact reference solver to quantify any ACO optimality gap. Until such cases exist, additional ACO seeds do not add route-order evidence.

5 Conclusion

The updated system has a clear two-stage interface: the MLP produces four risk scores, a fixed 0.70 filter selects non-empty warehouses, and single-drone ACO builds a distance-based route over only those warehouses. The ACO objective penalises unvisited active warehouses before distance, and payload is divided proportionally after route construction. This description matches the current configuration and implementation; earlier multi-drone, urgency-weighted, and daily-battery formulations are not part of the reported method.

The controlled rerun provides evidence that, for Warehouse 1 and the simulated target, the MLP is an accurate and stable risk-ranking model and that earlier risk-triggered dispatch can reduce overflow at an energy and distance cost. The 0.70 MLP-ACO policy reduced overflow by 11.38% relative to current-state routing while satisfying every checked constraint. However, positive prediction support is absent for Warehouses 2–4 and no interval exercises multi-stop routing. The defensible conclusion is therefore limited to prediction and dispatch filtering; the current experiment does not demonstrate that ACO solves a meaningful route-order problem.

6 Reproducibility Checklist

- Dataset file: `wepastacks_inbound_clean.csv`.
- Training recreated `model_outputs_single_drone`, including the saved model, predictions, metrics, calibration, threshold, seed-stability, ablation, and error-analysis artifacts.
- Prediction evidence includes baseline, support, threshold, confusion-matrix, and per-warehouse tables and figures.
- Command-line training entry point: `run_training.py`.
- Integrated experiment configuration: `config/experiment.yaml`.
- Routing entry point: `run_experiment.py`.
- Test command: `PYTHONPYCACHEPREFIX=/tmp/assignment2_pycache.venv/bin/python-munittestdiscover-stests-v`.
- Current verification: 21 tests passed in the available virtual environment; rerun immediately before submission.
- Routing recreated `outputs_single_drone` for the complete held-out period and all configured seeds.
- The run records ten seeds for each stochastic ACO policy and reports that the maximum simultaneous active-warehouse count is one.

A Configured Drone Experiment

Table 10: Simulated warehouse coordinates and capacities

Node	x (km)	y (km)	Capacity (cargo units)
Central depot 0	0	0	Not applicable
Warehouse 1	3	4	200
Warehouse 2	-5	2	80
Warehouse 3	6	-3	80
Warehouse 4	-4	-6	80

Table 11: Configured single simulated drone

Drone	Payload	Battery	Energy/km	Maximum route
1	30	30.0	1.0	30 km

Battery is reset before every 15-minute interval. This represents replacement or charging at the depot and is not an optimised charging schedule. The values in this appendix are experimental assumptions and must not be attributed to the WEPASacks warehouse.

B Metric Definitions

For true positives TP , false positives FP , true negatives TN , and false negatives FN :

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad (18)$$

$$\text{Specificity} = \frac{TN}{TN + FP}, \quad \text{F1} = \frac{2 \text{Precision Recall}}{\text{Precision} + \text{Recall}}, \quad (19)$$

$$\text{Balanced Accuracy} = \frac{\text{Recall} + \text{Specificity}}{2}. \quad (20)$$

The Brier score is the mean squared difference between a predicted score and the binary target. The held-out value is 0.0122. The accompanying reliability plot shows weak support and deviations in middle-score bins, so this value alone is not treated as proof of full calibration.

References

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.
- [2] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of AISTATS*, pages 249–256, 2010.
- [3] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- [4] M. Dorigo and L. M. Gambardella. Ant colony system: A cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, 1(1):53–66, 1997.
- [5] T. Saito and M. Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015.

- [6] WEPASacks project documentation. *Dataset Analysis Report, MLP Training Specification, MLP Training Report, Single-Drone MLP-ACO Implementation Guide, and Assessment Criteria*. Internal assignment documents, 2026.